Strangler Fig Pattern

Category: Migration / modernization **One-liner:** Replace an old system gradually by putting a router in front of it and moving one capability at a time to the new system, instead of rewriting everything and switching over in one go.

Part 0 — Concepts you need first

What is a monolith?

A **monolith** is an application built and deployed as one unit: one **codebase** (all its source files), one **artifact** (the runnable package the build produces, e.g. a `.jar`), one **process** (a running program with its own memory). Changing one line in reporting means rebuilding and **redeploying** (putting a new version live) *everything*, including payments.

Monoliths are not evil — for a small application they're usually the right choice. Problems start when it grows large and many teams must change it at once. (Architectural styles compared: § Architectural styles.)

What is a legacy system?

Old software that is still running because it does something essential. Nobody keeps unimportant old software running. That creates the trap this chapter is about: the system is **too risky to keep** (fragile, expensive) and **too risky to replace** (load-bearing).

What is a "big-bang rewrite"?

Build a complete replacement, then switch everything over on one date. It fails often, predictably: while you spend eighteen months building, the old system keeps changing (fixes, features, regulation) — you're rebuilding a moving target. On switchover day you discover the hundred undocumented behaviours, with no way back.

What is a proxy or router (in this context)?

Software **between** a client and one or more servers; the client talks only to the proxy, which decides where the request goes. **The client cannot tell the difference** — and that property is the entire foundation of the pattern: if clients can't distinguish the middle layer from the real system, you can change what's behind it whenever you like.

What is a feature flag?

A configuration switch that changes behaviour **without deploying new code**:

```
if (flags.isEnabled("use-new-implementation")) {
    return newImplementation(x);
}
return oldImplementation(x);
```

You change behaviour by editing configuration — and change it *back* in seconds if something breaks. Flags are what make the migration reversible.

Part 1 — Where the name comes from

The **strangler fig** seeds in the branches of a host tree, grows roots down and branches up around it, and eventually the host dies and rots away, leaving the fig standing in its shape. Two properties carry over:

1. **The fig grows around the host while the host is still alive** — both coexist for a long time.

2. **The transition is gradual and continuous** — there is no single moment when the tree becomes a fig.

In software: the new system grows *around* the legacy system, which keeps serving real **production** traffic (live users, not test environments) the whole time it's being displaced.

Part 2 — The problem in detail

Why legacy systems are painful to live with

Pain	What it means in practice
Technical debt	Years of rushed patches by people who've left; nobody is confident what a change affects. ("Debt": a shortcut now that charges interest later as slower change.)
Obsolete technology	The **stack** (languages, frameworks, databases, servers) is outdated: hard to hire for, no vendor **patches** (bug/security fixes), **libraries** with bugs that will never be fixed
Integration issues	Modern tools don't connect easily → **silos of information**: data manually exported and duplicated
Scalability concerns	Built for its era's load; may assume one server; **not cloud-ready** — can't simply run as more copies on the **cloud** (rented on-demand servers: AWS, Azure, Google Cloud) when traffic grows
Costly maintenance	Every change takes longer; fewer people understand the code
Security vulnerabilities	Predates current practice: weak password storage, no protection against now-standard attacks, an **OS** that no longer gets patches

Why legacy systems are painful to replace

- **Critical operations** — disruption isn't "the site is slow", it's "we cannot take orders today".
- **Migration risks** — moving data risks **loss or corruption**; old **schemas** (table/column structure) hold years of irregularity: rule-violating records, reused fields, **encoding** inconsistencies (garbled accents/symbols).
- **Training and adoption** — staff who were fast on the old system are suddenly slow, and resist.
- **High costs** — building a replacement *and making it as reliable as the old one*. The old system's reliability came from a decade of production bug-fixing your new one hasn't had.
- **Unknown unknowns** — undocumented behaviours users depend on.

> This last one kills rewrites. You can't specify behaviour nobody wrote down. You discover it at switchover: "the report used to round differently and now our reconciliation breaks." A rewrite finds them all at once, on the worst possible day.

Part 3 — The solution

Don't replace the system. **Strangle it.** Build the new system alongside the old one, put a routing layer in front of both, migrate one capability at a time, and flip the router as each piece proves itself in production. When no traffic reaches legacy, delete it.

Why this beats a rewrite

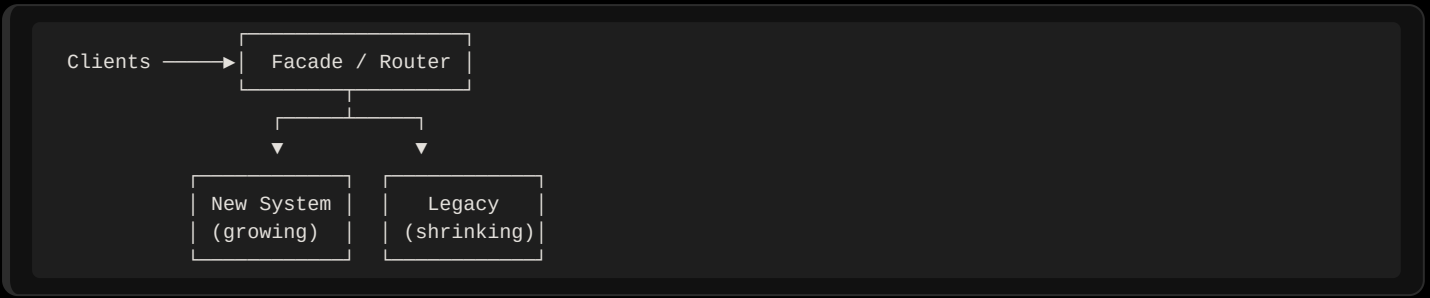
- **Less risky** — every step is individually revertible; if the new checkout breaks, route checkout back to legacy in seconds.
- **Flexible prioritisation** — migrate by **urgency or business need**, not the dependency order a rewrite forces. Most painful module first.

- **No big-bang releases** — **continuous delivery** (always releasable, small frequent changes) instead of one catastrophic **cutover** date (when traffic switches from old to new).
- **Smooth transition** — users may not notice; each piece is tested before it carries real traffic.
- **Early feedback loops** — real users hit new code early, while problems are cheap.
- **Technology flexibility** — not locked into what was fashionable at kickoff, which matters when a migration takes two years.

Part 4 — Architecture

The Facade Interface — the gatekeeper

Every request hits the facade first, and it makes one decision: **has this functionality been migrated yet?** Yes → new system; no → legacy.



The critical property: **clients are unaware there are two systems.** They see one address, one **API** (the requests a system accepts and the responses it returns), one system. ("Facade" = the Facade design pattern: a simple unified front over something complicated — here, two entire systems.)

Layered, phased migration

Peel the legacy system layer by layer:

1. **Identify a piece of functionality that can be cleanly isolated** and redirected.
2. **Rebuild it** in the new system.
3. **Test it fully.**
4. **Flip the facade** for that functionality only; everything else still goes to legacy.
5. **Repeat.**

Step 1 is the hard one. "Cleanly isolated" means it doesn't share **mutable state** (data that changes at runtime) with everything else. Most legacy systems have one giant shared database, so finding a genuine **seam** (a boundary with few connections across it) is the real work. Look for capabilities that read a lot and write little, or own a table nothing else writes to.

Part 5 — The four implementation steps

1. **Identify entry points** — choose what to replace and build new services for it.
2. **Route traffic** — a router/proxy directs each request to old or new based on what's migrated.
3. **Incrementally build and replace** — slowly displace old functionality.
4. **Retire the old** — once everything is taken over, **decommission** (permanently shut down) the legacy system.

Part 6 — Worked example 1: routing a single function

Replace just `getUserDetails` in an old user-management system:

```python
class UserServiceRouter:
    def get_user_details(self, user_id):
        if self.use_new_service(user_id):
            return NewUserService().get_user_details(user_id)
        else:
            return OldUserService().get_user_details(user_id)

    def use_new_service(self, user_id):
        # Logic deciding which service to use.
        # For simplicity: use the new service for even user IDs.
        return user_id % 2 == 0
```

- `UserServiceRouter` is the facade — **callers use it, never the two services directly**, so the switch is invisible.
- `get_user_details` has the same **signature** (name, parameters, return type) as the underlying services: a **drop-in replacement**.
- `use_new_service` is the **decision function** — the entire migration policy lives in this one method. Even IDs ≈ 50% of users.

Why the even-ID trick is smarter than it looks

It's a **percentage rollout dial**. A **hash function** turns an input into a number in a scrambled but repeatable way (same input → same output, outputs spread evenly), so `hash(user_id) % 100` puts each user in a fixed bucket 0–99. You choose *how much* new:

```python
return hash(user_id) % 100 < 5     # 5% of users
return hash(user_id) % 100 < 50    # 50% of users
return True                        # everyone
```

It's **consistent per user**: user 42 always gets the same implementation. If users flipped randomly per request, any behavioural difference would look like an unreproducible intermittent bug. Widen the condition as confidence grows until the old path is **dead code** (can never run) — then delete it.

Part 7 — Worked example 2: a facade over a banking system

Legacy and new systems expose **identical signatures** — deliberately, because the facade can only substitute one for the other if they're interchangeable:

```java
public class LegacySystem {
    public void createAccount(String name) { /* old logic */ }
    public void deposit(int amount)        { /* old logic */ }
    public void withdraw(int amount)       { /* old logic */ }
    public int  checkBalance()             { /* old logic */ }
}
// NewSystem: the same four methods with /* new logic */
```

The facade — the class the outside world talks to — holds both and directs each request:

```java
public class BankingFacade {
    private LegacySystem legacySystem;
    private NewSystem newSystem;

    public BankingFacade(LegacySystem legacySystem, NewSystem newSystem) {
        this.legacySystem = legacySystem;
        this.newSystem = newSystem;
    }

    public void createAccount(String name) {
        if (newSystemReadyFor("createAccount")) {
            newSystem.createAccount(name);
        } else {
            legacySystem.createAccount(name);
        }
    }

    // deposit(...) and checkBalance() follow the same if/else shape

    private boolean newSystemReadyFor(String functionality) {
        // Check whether the new system is ready for this functionality
        return false;   // placeholder — starts false for everything
    }
}
```

How the migration actually proceeds

At the start `newSystemReadyFor` returns `false` for everything: **100% of traffic goes to legacy**. Deploy the facade first and verify it changes nothing *before* migrating anything — if something breaks then, you know it's the facade, not new code.

Then rebuild `createAccount`, test it, and make `newSystemReadyFor("createAccount")` return `true`: account creation goes to the new system while deposits, withdrawals and balance checks stay on legacy. Repeat per capability.

The key structural insight: the only thing that changes at each cutover is what `newSystemReadyFor` returns. Clients don't change; the two systems don't change. In reality that method reads a **feature flag**, so flipping a capability needs no deployment — and flipping it *back* takes seconds.

Part 8 — Key insights: the proxy layer is everything

The router is on the critical path for **every request**, so it must be robust. Three kinds of routing logic:

- **Feature flags** — turn new functionality on/off; the superpower is **instant rollback** (back to known-good behaviour) with no deploy or rebuild.
- **User segmentation** — **A/B testing** (two variants to different groups, compare results) or **phased rollouts** (growing share of users): internal staff → 1% → 10% → everyone, or a small market first.
- **API versioning** — route on version **headers** (request metadata, e.g. `Accept-Version: 2`) or **paths** (the URL part after the domain) — `/v1/orders` to legacy, `/v2/orders` to new.

Part 9 — Trade-offs and what actually goes wrong

- **Running two systems at once** — for what may be years: two **deployment pipelines** (automated build-test-release), two **on-call rotations**, two sets of **dashboards**.
- **The facade is a single point of failure** — everything flows through it, so it needs **high availability** (redundant copies) and a careful **latency budget** (max delay each component may add — see § Latency): whatever it adds, every request pays.
- **An extra hop** — one more network trip on every request, however small.

- **Data consistency — the hard part the pattern doesn't solve.** If old and new share or split data you need a strategy (shared database, sync, or **dual writes** — the app writing to two databases). See Part 13D.

  > Why: migrate "create account" but leave "check balance" on legacy, with separate databases, and accounts created in the new system are invisible to legacy's balance check. Teams usually keep **one shared database** during migration and split it later — a second strangler migration in its own right.

- **Duplicated logic** — while a capability is in flight, the same rule lives in two places; change both or they diverge.
- **Migration fatigue — the most common real-world failure.** The pattern makes it *safe* to stop halfway, so organisations do: the urgent 60% migrates, the pain fades, priorities shift, and both systems run forever — worse than either alone. **Set a decommission date up front and treat it as a real deliverable.**

Part 10 — Common beginner misconceptions

- **"The new system must be microservices."** (**Microservices**: many small, independently deployable services, each owning one capability.) No — Strangler Fig is *how* you migrate, not *what to*. You can strangle a monolith into a better monolith.
- **"You migrate the database first."** Usually the opposite: data is the riskiest part, so keep the shared database, move behaviour first, split data later.
- **"The facade is temporary."** Often it becomes the permanent API Gateway. Fine, if deliberate.
- **"Delete old code as soon as a capability migrates."** It's your rollback. Leave it unreachable but intact until the flag has been at 100% for a sensible period.
- **"Only for ancient systems."** It works whenever you replace something in use — a payment provider, a search engine, a cloud.

Part 11 — Check your understanding

1. Why is the strangler fig a better metaphor than "demolish and rebuild"?
2. What single method in the `BankingFacade` example controls the entire migration?
3. Why route by a *consistent* function of user ID rather than randomly per request?
4. Why deploy the facade before migrating anything?
5. What problem does the pattern explicitly *not* solve?
6. What's the most common organisational failure mode, and the countermeasure?

(Answers: 1 — both systems coexist and the transition is gradual and reversible; 2 — `newSystemReadyFor`; 3 — a user must get consistent behaviour, or differences look like intermittent bugs; 4 — to verify the facade itself is harmless before adding a second variable; 5 — data consistency between old and new; 6 — stalling halfway, prevented by committing to a decommission date.)

Part 12 — Interview soundbite (30 seconds)

> "Strangler Fig replaces a legacy system without a big-bang rewrite. You put a facade or proxy in front of the old system so clients only talk to that, then rebuild one capability at a time behind it. The facade routes each request to the new service or legacy depending on what's migrated — usually via feature flags or user segmentation, giving percentage rollouts and instant rollback without a deploy. Traffic to legacy drops to zero and you delete it. It's named after a tree that

> grows around its host until the host rots away. It beats a rewrite because a rewrite chases a moving target and discovers all the undocumented legacy behaviour on switchover day, whereas this surfaces problems one capability at a time while you still have a way back. Trade-offs: running both systems for a long time, the facade as a critical single point of failure, and data consistency between old and new, which the pattern doesn't solve. The biggest practical risk is organisational — it's safe to stop halfway, so teams do; commit to a decommission date up front."

Part 13 — Implementation

A. Runnable demo

Full source: `demos/StranglerFigDemo.java`

```
cd demos && ./gradlew runCh02
```

(`./gradlew` is the **Gradle wrapper**: it downloads and runs Gradle, the Java build tool used for every demo in this book.)

The core is the flag check that makes rollout **consistent per user**:

```
boolean useNewSystem(String capability, String userId) {
    int percent = rolloutPercent.getOrDefault(capability, 0);
    if (percent <= 0)   return false;
    if (percent >= 100) return true;
    int bucket = Math.floorMod(userId.hashCode(), 100);   // stable per user
    return bucket < percent;
}
```

`Math.floorMod`, not `%`, because `hashCode()` can be negative and `%` would give a negative bucket.

B. What you should see

```
=== Step 1: facade deployed, nothing migrated ===
  LEGACY:created:alice
  >> all traffic still on LEGACY - facade proven harmless

=== Step 3: same user always gets the same system ===
  user-7 -> LEGACY:created:acct     (x3, never flips)

=== Step 4: widen the rollout ===
    [flags] createAccount -> 25%  ->  actual: 253/1000 users
    [flags] createAccount -> 50%  ->  actual: 493/1000 users
    [flags] createAccount -> 100% ->  actual: 1000/1000 users

=== Step 5: problem found - INSTANT rollback ===
  user-7 -> LEGACY:created:acct
  >> no deploy, no rebuild: config change only
```

Step 3 proves consistency — user-7 never flips. The percentages aren't exact (25% gave 253/1000, 50% gave 493) because **hash distribution is only approximately uniform** at these sample sizes. That's fine: you're **canarying** (sending a small slice of real traffic to the new version first), not metering.

C. Production — routing at the proxy

In reality the facade is usually infrastructure, not application code.

Nginx — route by path (a **reverse proxy** forwards client requests to servers; an `upstream` block names a backend group):

```
upstream legacy_system { server legacy.internal:8080; }
upstream new_system    { server orders-v2.internal:8080; }

server {
    location /api/orders  { proxy_pass http://new_system;    }   # migrated
    location /api/billing { proxy_pass http://legacy_system; }   # not yet
    location /            { proxy_pass http://legacy_system; }   # default: legacy
}
```

Istio — percentage rollout without touching application code (Istio is a **service mesh**, configured in YAML; see Sidecar):

```
apiVersion: networking.istio.io/v1beta1
kind: VirtualService
metadata:
  name: orders
spec:
  hosts: [orders]
  http:
    # Internal staff always get the new system - your first canary group
    - match:
        - headers:
            x-user-group: { exact: internal }
      route:
        - destination: { host: orders, subset: v2 }
    # Everyone else: 5% to the new system
    - route:
        - destination: { host: orders, subset: v1 }
          weight: 95
        - destination: { host: orders, subset: v2 }
          weight: 5
```

Feature flags in application code — LaunchDarkly (hosted), Unleash (open source), or a table in your config store (Config Externalization):

```
if (flags.isEnabled("orders-v2", userContext)) {
    return newOrderService.create(order);
}
return legacyOrderService.create(order);
```

D. The part the pattern doesn't solve: data

Three strategies, in increasing difficulty:

1. **Shared database (start here).** Both systems use the same database and old schema — no sync problem. Split the data later as a separate migration.
2. **Dual writes.** The new system writes to both databases. Easy to describe, hard to get right: if the second write fails the copies diverge, and no **transaction** (all-or-nothing group of changes) spans both. (Same root problem as the Transactional Outbox.)
3. **Change Data Capture (CDC).** Watch every insert/update/delete in the legacy database's **transaction log** (its append-only record of committed changes) with **Debezium** and stream them into the new one. The new data is **eventually consistent** (may lag briefly, then catches up), and **the legacy system needs no modification**:

```
# Debezium connector: legacy Postgres -> Kafka -> new service
connector.class: io.debezium.connector.postgresql.PostgresConnector
database.hostname: legacy-db
table.include.list: public.orders,public.customers
plugin.name: pgoutput
```

(Kafka, the event log in the middle, is covered in Event-Driven.)

E. Implementation pitfalls

1. Random rather than consistent routing. `Math.random() < 0.05` sends the *same user* to different systems on consecutive requests, so any difference becomes an unreproducible bug. Always hash a stable identifier.

2. Deploying the facade and migrating a capability in one release. If it breaks you can't tell which change did it. Facade first, confirm no-op, then migrate.

3. Deleting legacy code the moment traffic hits zero. It's your rollback. Keep it until the flag has been at 100% for a full business cycle.

4. No comparison stage. Before flipping traffic, run **shadow mode**: send the request to both, return the legacy response, log differences. This finds undocumented behaviour *before* users do. The new system runs **asynchronously** (on another thread, so the user never waits) and records **metrics** (monitoring counters):

```
Result legacyResult = legacy.process(request);
executor.submit(() -> {                          // async, never blocks the user
    try {
        Result newResult = modern.process(request);
        if (!newResult.equals(legacyResult)) {
            metrics.increment("shadow.mismatch");
            log.warn("Divergence for {}: {} vs {}", request.id(), legacyResult, newResult);
        }
    } catch (Exception e) { metrics.increment("shadow.error"); }
});
return legacyResult;                             // user always gets the proven answer
```

5. Flags that outlive the migration. Each finished migration leaves a dead **conditional**. Make flag removal part of "done", or the codebase fills with permanent forks.

6. No decommission date. The organisational failure mode from Part 9. Put it in the plan.

Related patterns

- **API Gateway** — the facade is very often literally an API Gateway
- **BFF** — can bridge new frontends to legacy systems during migration
- **Config Externalization** — where the feature flags driving routing actually live